CS 747: Programming Assignment 2

Akash Sansugu Palaniswami
21D171001

March 2025

1 Task 1: MDP Planning

1.1 Introduction

The objective of this task was to successfully formulate the procedure of linear programming and Howard's policy iteration. for a given Markov Decision Problem in order to find the optimal policy π^* and the ideal value function V^* for the MDP. The other objective was to successfully calculate the value function for each state given a particular policy.

1.2 Extracting MDP Information

A dictionary was created to store all the information of the MDP, with the keys

- "num_states", which contains the total number of states
- "num_actions", which contains the total number of possible actions
- "end_states", to store the states at which the mdp is terminal and doesn't continue
- "transitions", which is a list with each element containing the start state, action, end state, reward, and probability of each transition
- "type", which lets us know if the mdp is continuous or episodic
- "discount", which gives us the discount factor or γ

This information was extracted by parsing through the mdp text file, line by line using the split function and seeing what the first element of each line is.

```
def main():
    mdp={"num_states":0, "num_actions":0,"end_state":[], "transitions": [], "type": "", "discount":0}
    file=open(mdp_path, "r")
    read=file.readlines()
    for line in read:
        x = line.strip().split()
        if x[0]=="numStates":
            mdp["num_states"]=int(x[1])
        elif x[0]=="numActions":
            mdp["num_actions"]=int(x[1])
        elif x[0]=="end":
            mdp["end_state"]=x[1:]
        elif x[0]=="transition":
            s=int(x[1])
            a=int(x[2])
            s_dash=int(x[3])
            r=float(x[4])
            p=float(x[5])
            mdp["transitions"].append((s,a,s_dash,r,p))
        elif x[0]=="mdptype":
            mdp["type"]=x[1]
        elif x[0]=="discount":
            mdp["discount"]=float(x[1])
```

Figure 1: Code for Extracting MDP Information

1.3 Howard's Policy Iteration (HPI)

1.3.1 Theory

This is done by first assigning a random policy to each state and then calculating the value function, V, for each state. After this, we iterate through each state and calculate Q for all possible actions. The action that provides maximum value of Q is then taken as the action for that state, during the next iteration. This process continues until we finally end up with the maximum value of Q already being the chosen action for each and every state.

$$V^{\pi} = (I - \gamma T^{\pi})^{-1} R^{\pi}$$

$$Q^{\pi}(s,a) = \sum_{s' \in \mathcal{S}} T(s,a,s')\{R(s,a,s') + \gamma V^{\pi}(s')\}.$$

In the above equations T^{π} and R^{π} represent the transition and reward matrices of the mdp, and are used to calculate the value functions of all the states.

1.3.2 Implementation

- We defined 2 different functions for implementing HPI for continuous and episodic mdp's. It is decided which function to call based on the end state given by the mdp, if it is -1 we know it is continuous and episodic otherwise

- The first for loop in Figure 3: is to assign initial actions to the states, by checking what is the first action for a state that exists and assigning that action.

```
elif algo=='hpi':
    if int(mdp["end_state"][0])<0:

        a,b=HPI(mdp)
        for i in range(mdp["num_states"]):
            print(a[i],b[i])
    else:
        a,b=HPI_episodic(mdp)
        for i in range(mdp["num_states"]):
            print(a[i],b[i])
```

Figure 2: Choosing between Continuous and Episodic MDPs

```
def HPI(mdp):

    arr=np.zeros(mdp["num_states"])
    I=np.eye(mdp["num_states"])
    for i in range(mdp["num_states"]):

        target_first_value = i
        l=[lst for lst in mdp["transitions"] if lst[0] == target_first_value]
        arr[i]=l[0][1]
    k=10
    while k>0:
        T=np.zeros((mdp["num_states"],mdp["num_states"]))
        R=np.zeros(mdp["num_states"])
        for i in range(mdp["num_states"]):
            z=0
            for j in range(mdp["num_states"]):
                l=[lst for lst in mdp["transitions"] if lst[0] == i and lst[1] == arr[i] and lst[2]== j]
                #print(l)
                if bool(l):
                    T[i][j]=l[0][4]
                    z=z+l[0][4]*l[0][3]
            R[i]=z
        V_1=np.linalg.inv(I-mdp["discount"]*T)
        V=np.dot(V_1,R)
        a=I-mdp["discount"]*T
        #V=np.linalg.solve(a,R)
        #print(V)
        #print(arr)
```

Figure 3: Calculation of Values States

```
    my_dict = {i: None for i in range(mdp["num_states"])}
    e=0.000001
    for i in range(mdp["num_states"]):
        v_temp=V[i]
        for j in range(mdp["num_actions"]):
            b=0
            l=[lst for lst in mdp["transitions"] if lst[0] == i and lst[1] == j]
            for x in l:
                s=x[2]
                b=b+ x[4]*(x[3]+mdp["discount"]*V[s])
                #print(i,b,V[i])
            if b-V[i]>e:
                my_dict[i]=1
            if b>v_temp:
                arr[i]=j
                v_temp=b
    #k=k-1
    #print(my_dict)
    if all(value is None for value in my_dict.values()):
        break
return V,arr
```

Figure 4: Updating Actions

- Inside the while loop, we have a nested for loop initially to transition and rewards matrix, which is then used to calculate the value functions of all the states.

- In Figure 4, we first create a dictionary with number of keys the same as the number of states. Each value is initialized to None, and it will be changed to 1 if there exists a action other than the current action that results in a value of Q higher than V.

- Each actions are then tested for a test and updated if necessary. After updating, if all the values of the above dictionary my_dict are None, which means we have arrived at our optimal policy.

- The only difference in the code for an episodic function is in the formation of Transition and Reward matrices. We check if the given state

```
while k>0:
    T=np.zeros((mdp["num_states"],mdp["num_states"]))
    R=np.zeros(mdp["num_states"])
    for i in range(mdp["num_states"]):
        if str(i) in end_state:
            T[i:]=0
            R[i]=0
            continue
        z=0
        for j in range(mdp["num_states"]):
            l=[lst for lst in mdp["transitions"] if lst[0] == i and lst[1] == arr[i] and lst[2]== j]
            #print(l)
            if bool(l):
             T[i][j]=l[0][4]
             z=z+l[0][4]*l[0][3]
        R[i]=z
    V_1=np.linalg.inv(I-mdp["discount"]*T)
    V=np.dot(V_1,R)
    a=I-mdp["discount"]*T
```

Figure 5: Addition in Function for Episodic MDP

is an end state and if it is we directly initialize the respective values to 0. This is shown in Figure 5.

1.3.3 Linear Programming (LP)

1.3.4 Theory

When we calculate value functions for each state, it is always in terms of value functions of other states. However we can see that the value functions are always linear in nature. This means that we acquire a system of x equations in x variables, where x is the number of states. This means that there is almost always an optimal solution. To solve linear systems we use the PULP library in python. We use the Bellman's Equation to obtain the necessary constraints, and the objective function is to maximise our reward.

$$\textbf{Maximise} \quad \left(-\sum_{s \in \mathcal{S}} V(s) \right)$$

subject to

$$V(s) \geq \sum_{s' \in \mathcal{S}} T(s, a, s') \{ R(s, a, s') + \gamma V(s') \}, \quad \forall \mathbf{s} \in \mathcal{S}, \mathbf{a} \in \mathcal{A}.$$

1.3.5 Implementation

- We first define our linear programming problem as an minimization problem, and create variables corresponding to each state.

```
lp=LpProblem("MDP",LpMinimize)
var_keys=np.arange(mdp["num_states"])
x=LpVariable.dicts('Value_Functions',var_keys,lowBound=None)
a=0
for i in range(mdp["num_states"]):
    a=a+x[i]
#a=(-1)*a
lp += a

#print(mdp["end_state"])
for i in range(mdp["num_states"]):
    if str(i) in mdp["end_state"]:
        lp+=(x[i]==0)
        continue
    for c in range(mdp["num_actions"]):
        b=0
        target_first_value = i
        target_second_value = c
        l=[lst for lst in mdp["transitions"] if lst[0] == target_first_value and lst[1] == target_second_value]
        if len(l)!=0:

            for j in l:
                if j[0]==i and j[1]==c:
                    b=b+j[4]*(j[3]+mdp["discount"]*x[j[2]])
            lp +=(x[i]>=b)
```

Figure 6: Objective Function and Constraints

- The first for loop defines our objective function as the sum of all variables that we defined. Instead of maximizing the negative of this sum, we choose to minimize the sum.
- The nested for loop is used to define the constraints of the problem. Initially we check if any of the states are terminal, and directly put the constraint as, its value function equal to zero.
- We then iterate through all actions for each state, and retrieve the transitions for which we start from the respective state and use the respective action. Using each state action pair we obtain a constraint.

```
V_need = np.zeros(mdp["num_states"])
for i in range(mdp["num_states"]):
    V_need[i]=value(x[i])
#print(V_optimal)
#print(values[:5])
actions=[]
e=0.00001
for i in range(mdp["num_states"]):
    if str(i) in mdp["end_state"]:
        actions.append(0)
    min=sys.float_info.max
    c_real=10000000
    for c in range(mdp["num_actions"]):
        #print("C is", c)
        b=0
        target_first_value = i
        target_second_value = c
        l=[lst for lst in mdp["transitions"] if lst[0] == target_first_value and lst[1] == target_second_value]
        for j in l:
            b=b+j[4]*(j[3]+mdp["discount"]*values[j[2]])
        #print("A",abs(values[i]-b))
        d=abs(V_need[i]-b)
        if d<=min:
            min=d
            c_real=c
    actions.append(c_real)
h=0
#print(value(lp.variables))
for i in range(mdp["num_states"]):
    print(V_need[i],actions[h])
    h=h+1
```

Figure 7: Identifying Ideal Value Functions and Optimal Policy

- The optimal value functions for each state are stored in an array.
- To find the optimal policy, we first create an empty array to store the actions. We first directly assign a action of 0 for a terminal state.
- We then iterate through all actions for each state and calculate its value function based on the optimal value functions that we have already calculated. We compare how close it is with the required value function, and whichever comes out to be closest we take that as the optimal action. This works because we know that actually we should obtain the exact value of required value function for the optimal policy alone.

1.4 Policy Evaluation

To evaluate the value function based on a given policy, we utilize the matrix formula that we made use of in Howard's Policy Iteration

```
if (policy != ''):

    pol=[]
    file=open(policy, "r")
    read=file.readlines()
    for line in read:
        x = line.strip().split()
        pol.append(int(x[0]))
    I=np.eye(mdp["num_states"])
    T=np.zeros((mdp["num_states"],mdp["num_states"]))
    R=np.zeros(mdp["num_states"])
    for i in range(mdp["num_states"]):
        z=0
        for j in range(mdp["num_states"]):
            l=[lst for lst in mdp["transitions"] if lst[0] == i and lst[1] == pol[i] and lst[2]== j]
            #print(l)
            if bool(l):
             T[i][j]=l[0][4]
             z=z+l[0][4]*l[0][3]
        R[i]=z
    V_1=np.linalg.inv(I-mdp["discount"]*T)
    V=np.dot(V_1,R)

    h=0

    for var in range(mdp["num_states"]):
        print(V[var],pol[var])
        h=h+1
        if h>=len(pol):
            break
```

Figure 8: Policy Evaluation

- First we check that the input for policy is not empty, so that we know we have to do policy evaluation instead of lp or hpi.
- Next, we extract the actions for each state from the .txt file and store them in an array.
- Transition and reward matrices are calculated, and then value functions are calculated using the same equation as we used in Howard's Policy Iteration

2 Task 2: MDP-based Gridworld Navigation

2.1 Encoder

```
def main():

    file=open(grid_path, "r")
    #line_count = sum(1 for _ in enumerate(file))
    read=file.readlines()
    line_count=0
    for line in read:
        x = line.strip().split()
        #print(x)
        l=len(x)
        line_count=line_count+1
    map=np.zeros((line_count,l))
    file=open(grid_path, "r")
    read=file.readlines()
    i=0
    num_states=0
    num_actions=4
    discount=0.95
    mdp_type='episodic'
```

Figure 9: Finding Dimensions of Map

```
for line in read:
    line = line.strip().split()
    #print(line)
    for j in range(l):
        if line[j]=='W':
            map[i][j]=0
        elif line[j]=='_':
            map[i][j]=1
            num_states+=1
        elif line[j]=='s':
            map[i][j]=2
            num_states+=1
            start=l*i+j
        elif line[j]=='k':
            map[i][j]=3
            num_states+=1
            key=(i,j)
        elif line[j]=='d':
            map[i][j]=4
            num_states+=1
            door=(i,j)
        elif line[j]=='g':
            map[i][j]=5
            end_state=(i,j)
            num_states+=1
    i=i+1
```

Figure 10: Updating Actions

- First I opened the file and counted number of rows and columns, to be able to form a matrix of similar dimensions
- Filled in the matrix, map assigning numerical values to all symbols

- I created transitions with 5 elements.
- Initial element represents initial state, which contains the coordinates of the position and the direction that its facing, which is represented by an integer 0-4
- Second element represents the action
- Third element represents the end state which also contains the coordinates of the end point and the direction its facing

```
dict={"states":[],"actions":[0,1,2,3],"start":start,"key":key,"door":door}
for i in range(line_count):
    for j in range(l):
        if map[i][j]==1 or map[i][j]==2 or map[i][j]==3 or map[i][j]==4:
            dict["states"].append((i,j))
W=0
t=[]
for x in dict["states"]:

    for y in range(4):# 0 is up, 1 is right, 2 is down, 3 is left


        if y==0:
            for z in range(4):
                if z==0:
                    if (map[x[0]-1][x[1]]!=W):
                        if(map[x[0]-2][x[1]]!=W):
                            if(map[x[0]-3][x[1]]!=W):
                                t.append(((x,y),z,((x[0]-1,x[1]),y),0,0.5))
                                t.append(((x,y),z,((x[0]-2,x[1]),y),0,0.3))
                                t.append(((x,y),z,((x[0]-3,x[1]),y),0,0.2))
                            else:
                                t.append(((x,y),z,((x[0]-1,x[1]),y),0,0.5))
                                t.append(((x,y),z,((x[0]-2,x[1]),y),0,0.5))
                        else:
                            t.append(((x,y),z,((x[0]-1,x[1]),y),0,1))
                elif z==1:
                    t.append(((x,y),z,((x[0],x[1]),y+3),0,0.9))
                    t.append(((x,y),z,((x[0],x[1]),y+2),0,0.1))
                elif z==2:
                    t.append(((x,y),z,((x[0],x[1]),y+1),0,0.9))
                    t.append(((x,y),z,((x[0],x[1]),y+2),0,0.1))
                else:
                    t.append(((x,y),z,((x[0],x[1]),y+2),0,0.8))
                    t.append(((x,y),z,((x[0],x[1]),y+1),0,0.1))
                    t.append(((x,y),z,((x[0],x[1]),y+3),0,0.1))
```

Figure 11: Making Transitions

- 4th element contains the reward, which is initally all kept as zero
- 5th elements represents the probability
- The variable y represents the inital direction while z is the action taken. Similarly all possibilities were taken into consideration

- I decided that the best course of action was to direct the agent towards the key if it had not been obtained yet and direct it to the door if it already had been acquired, so corresponding to this i assigned rewards for the key and door based on the circumstances
- I then outputted the required information into a .txt file in the required format as the mdp previously provided.

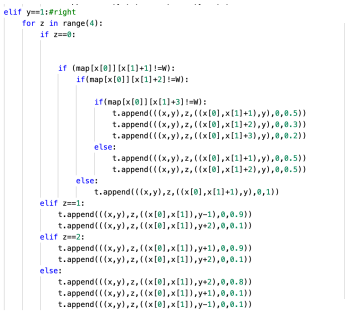

Figure 12:

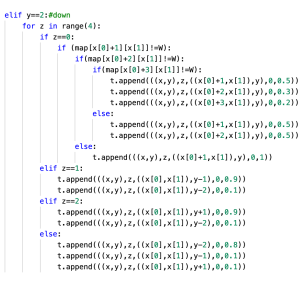

Figure 13:

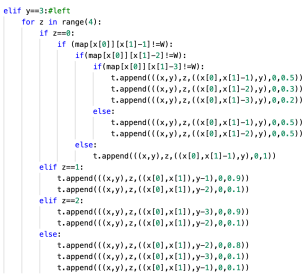

Figure 14:

```
if any(3 in row for row in map):
    target_value = key

    for i in range(len(t)):
        if t[i][2][0] == key:

            t[i] = ((((t[i][0][0][0], t[i][0][0][1]),t[i][0][1]), t[i][1], ((t[i][2][0][0], t[i][2][0][1]),t[i][2][1]), 1000, t[i][4])
else:
    target_value = key

    for i in range(len(t)):
        if t[i][2][0] == door:

            t[i] = ((((t[i][0][0][0], t[i][0][0][1]),t[i][0][1]), t[i][1], ((t[i][2][0][0], t[i][2][0][1]),t[i][2][1]), 1000, t[i][4])
with open("output.txt", "w") as file:
    print("numStates", num_states, file=file)
    print("numActions 4", file=file)
    print("end", end_state, file=file)
    for i in range(len(t)):
        print("transition", end=' ', file=file)
        for j in range(5):
            print(t[j],end=' ',file=file)
        print("\n",file=file)
    print("mdptype episodic", file=file)
    print("discount", 0.95, file=file)
```

Figure 15: Outputting into a .txt file

2.1.1 Issues with Decoder

- I was not able to successfully run my planner.py with this output obtained as the format of the mdp is slightly different, with the first element of the transition being a tuple which actually contained another tuple and an element, instead of just one element overall.

- Similar issues with with taking the end state